

Name: Harshita Singh
Class: D15C
Roll no. : 64
Subject: Machine Learning & Deep Learning

Experiment 10: Explore and implement Autoencoders for image denoising

1. Dataset Source

- **Dataset Name:** Fashion MNIST
- **Source:** tensorflow.keras.datasets.fashion_mnist
- **URL:** <https://github.com/zalandoresearch/fashion-mnist>
- **Context:** While MNIST contains digits, Fashion MNIST consists of Zalando's article images. It is used here to demonstrate denoising on more complex structural shapes (clothing items) than simple digits.

2. Dataset Description

- **Instances:** 70,000 grayscale images.
 - Training Set: 60,000 images.
 - Testing Set: 10,000 images.
- **Input Features:** 28x28 pixel grayscale images.
- **Target Variable:** In a Denoising Autoencoder (DAE), the target is the **original clean image** itself, not a label.
- **Characteristics:** Contains 10 categories (T-shirt, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, Ankle boot). Images are more complex than digits, making the reconstruction task more challenging.

3. Mathematical Formulation of the Algorithm

An **Autoencoder** is an unsupervised artificial neural network that learns how to efficiently compress and encode data, then learns how to reconstruct the data back from the reduced encoded representation to a representation that is as close to the original input as possible.

A **Denoising Autoencoder (DAE)** is a specific variant designed for robustness. Unlike a standard autoencoder that tries to reconstruct the input x from x , a DAE is trained to reconstruct the original clean input x from a corrupted version $\tilde{x}$. This forces the network to capture the "natural structure" of the data and ignore the statistical noise, effectively learning the manifold on which the data resides.

A. The Noise Injection Process

Before the data enters the network, a corruption process $C(\tilde{x}|x)$ is applied. We introduce stochastic noise (such as Gaussian noise) to the original input x to create a noisy version $\tilde{x}$:

$$\tilde{x} = x + \epsilon$$

Where:

- x : The original clean image.
- ϵ : Random noise sampled from a distribution (e.g., $\epsilon \sim \mathcal{N}(0, \sigma^2)$).
- $\tilde{x}$: The corrupted input fed into the encoder.

B. Encoder Function (Compression)

The encoder serves as a dimensionality reduction agent. It maps the high-dimensional noisy input $\tilde{x}$ into a low-dimensional **Latent Space** representation h .

$$h = f(W_1\tilde{x} + b_1)$$

- h : The latent vector (or "code") representing the compressed features.
- W_1 and b_1 : The weights and biases of the encoder layers.
- f : A non-linear activation function, such as **ReLU** ($f(z) = \max(0, z)$), which allows the network to learn complex spatial hierarchies.

C. Decoder Function (Reconstruction)

The decoder acts as a generative model that attempts to "project" the latent representation h back into the original input space. Its goal is to produce a reconstructed image $\hat{x}$ that is clean of noise.

$$\hat{x} = g(W_2h + b_2)$$

- $\hat{x}$: The reconstructed (denoised) output.
- W_2 and b_2 : The weights and biases of the decoder layers.
- g : The output activation function. In image processing, we typically use the **Sigmoid** function ($g(z) = \frac{1}{1+e^{-z}}$) to ensure the output pixel values remain normalized between 0 and 1.

D. The Latent Space (Bottleneck)

The latent space is the most critical part of the architecture. By restricting the number of neurons in the middle layers, we create a **bottleneck**. Mathematically, the dimensionality of h is much smaller than x ($\dim(h) \ll \dim(x)$). Because the network cannot pass all the information through this narrow gap, it is forced to discard the random noise (which is non-structured) and only preserve the most salient features (edges, textures, and shapes).

E. Loss Function (Mean Squared Error)

The objective of the Denoising Autoencoder is to minimize the reconstruction error. We measure the discrepancy between the reconstructed output $\hat{x}$ and the **original clean image** x (not the noisy $\tilde{x}$). The **Mean Squared Error (MSE)** is used:

$$\mathcal{L}(x, \hat{x}) = \frac{1}{N} \sum_{i=1}^N (x_i - \hat{x}_i)^2$$

Explanation of Terms:

- N : Total number of pixels in the image.
- x_i : The i^{th} pixel of the original clean image.
- $\hat{x}_i$: The i^{th} pixel of the denoised output image.
- $\mathcal{L}$: The loss value that the **Adam Optimizer** seeks to minimize during backpropagation.

4. Algorithm Limitations

- **Loss of Detail:** If the bottleneck (latent space) is too small, the reconstructed image may lose fine details and appear blurry.
- **Specific Noise Sensitivity:** An autoencoder trained to remove "Gaussian noise" may not perform well against "Salt and Pepper" noise or motion blur.
- **Overfitting:** If the capacity of the autoencoder is too high, it might just "memorize" the noise instead of learning to filter it.
- **Data Dependence:** The model can only denoise images similar to the ones it was trained on.

5. Methodology / Workflow

1. **Data Loading:** Retrieve the Fashion MNIST dataset.
2. **Preprocessing:** Normalize pixels to the $[0, 1]$ range.
3. **Noise Injection:** Artificially add random Gaussian noise to the images to create the "Noisy Input" set.

4. **Model Architecture:**
 - **Encoder:** Two Conv2D layers with MaxPooling to downsample the image.
 - **Decoder:** Two Conv2D layers with UpSampling2D to restore the image to 28x28.
5. **Compilation:** Use the Adam optimizer and Mean Squared Error (MSE) loss.
6. **Training:** Train the model using Noisy Images as **Input** and Clean Images as **Targets**.
7. **Evaluation:** Compare noisy images, original images, and denoised images side-by-side.

Python Code Implementation

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras import layers, models
from sklearn.metrics import mean_squared_error
import seaborn as sns

# =====
# 1. LOAD AND PREPROCESS DATA
# =====
print("--- Step 1: Loading Fashion MNIST Dataset ---")
# Load dataset (Targets are not needed for Autoencoders as the image
is the target)
(x_train, _), (x_test, _) =
tf.keras.datasets.fashion_mnist.load_data()

# Normalize pixel values to be between 0 and 1
x_train = x_train.astype('float32') / 255.
x_test = x_test.astype('float32') / 255.

# Reshape to (samples, height, width, channels)
x_train = np.expand_dims(x_train, axis=-1)
x_test = np.expand_dims(x_test, axis=-1)

print(f"Training data shape: {x_train.shape}")
print(f"Testing data shape: {x_test.shape}")

# =====
# 2. NOISE INJECTION (Corrupting the Data)
# =====
print("\n--- Step 2: Injecting Synthetic Gaussian Noise ---")
noise_factor = 0.4

# Add random normal noise scaled by the noise factor
```

```

x_train_noisy = x_train + noise_factor * np.random.normal(loc=0.0,
scale=1.0, size=x_train.shape)
x_test_noisy = x_test + noise_factor * np.random.normal(loc=0.0,
scale=1.0, size=x_test.shape)

# Clip the values to maintain the valid [0, 1] range
x_train_noisy = np.clip(x_train_noisy, 0., 1.)
x_test_noisy = np.clip(x_test_noisy, 0., 1.)

# =====
# 3. BUILD THE CONVOLUTIONAL AUTOENCODER
# =====
print("\n--- Step 3: Architecting the Denoising Autoencoder ---")

input_img = layers.Input(shape=(28, 28, 1))

# --- ENCODER ---
# Conv Layer 1: Downsampling to 14x14
x = layers.Conv2D(32, (3, 3), activation='relu',
padding='same')(input_img)
x = layers.MaxPooling2D((2, 2), padding='same')(x)
# Conv Layer 2: Downsampling to 7x7 (Latent Space Bottleneck)
x = layers.Conv2D(32, (3, 3), activation='relu', padding='same')(x)
encoded = layers.MaxPooling2D((2, 2), padding='same')(x)

# --- DECODER ---
# Conv Layer 3: Upsampling back to 14x14
x = layers.Conv2D(32, (3, 3), activation='relu',
padding='same')(encoded)
x = layers.UpSampling2D((2, 2))(x)
# Conv Layer 4: Upsampling back to 28x28
x = layers.Conv2D(32, (3, 3), activation='relu', padding='same')(x)
x = layers.UpSampling2D((2, 2))(x)
# Final Layer: 1 filter (Grayscale) with Sigmoid to ensure [0,1]
output
decoded = layers.Conv2D(1, (3, 3), activation='sigmoid',
padding='same')(x)

# Compile Model
autoencoder = models.Model(input_img, decoded)
autoencoder.compile(optimizer='adam', loss='mse')

autoencoder.summary()

```

```

# =====
# 4. MODEL TRAINING
# =====
print("\n--- Step 4: Training the Model (Mapping Noisy -> Clean) ---")

history = autoencoder.fit(
    x_train_noisy, x_train,
    epochs=15,
    batch_size=128,
    shuffle=True,
    validation_data=(x_test_noisy, x_test),
    verbose=1
)

# =====
# 5. PERFORMANCE METRICS & VISUALIZATION
# =====

# 1. Plot Training vs Validation Loss
plt.figure(figsize=(10, 5))
plt.plot(history.history['loss'], label='Training Loss (MSE)',
color='blue', linewidth=2)
plt.plot(history.history['val_loss'], label='Validation Loss (MSE)',
color='orange', linewidth=2)
plt.title('Denoising Autoencoder Learning Curve', fontsize=14)
plt.xlabel('Epochs')
plt.ylabel('Mean Squared Error')
plt.legend()
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()

# Predict (Denoise) images from the test set
decoded_imgs = autoencoder.predict(x_test_noisy)

# 2. Statistical Metrics
mse_noisy = mean_squared_error(x_test.flatten(),
x_test_noisy.flatten())
mse_denoised = mean_squared_error(x_test.flatten(),
decoded_imgs.flatten())

print("\n" + "="*40)
print(" PERFORMANCE EVALUATION TABLE")
print("="*40)
print(f"MSE (Original vs Noisy):      {mse_noisy:.4f}")

```

```

print(f"MSE (Original vs Denoised): {mse_denoised:.4f}")
print(f"Improvement Factor:          {mse_noisy/mse_denoised:.2f}x")
print("="*40)

# 3. Final Visual Comparison Grid
n = 8 # Number of items to display
plt.figure(figsize=(18, 10))

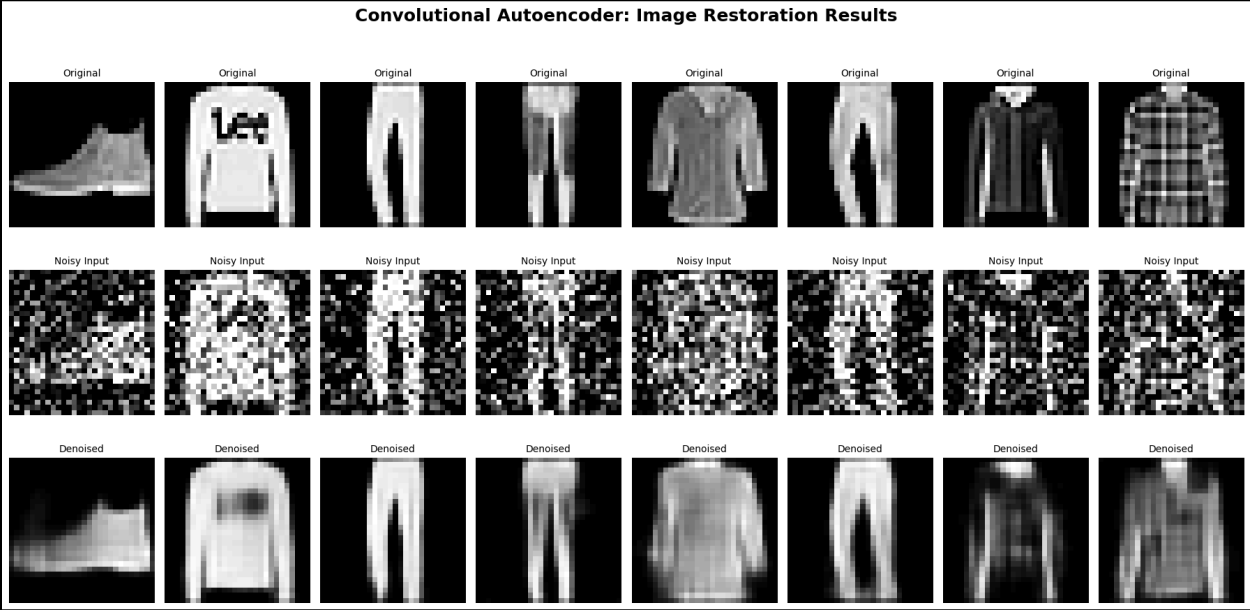
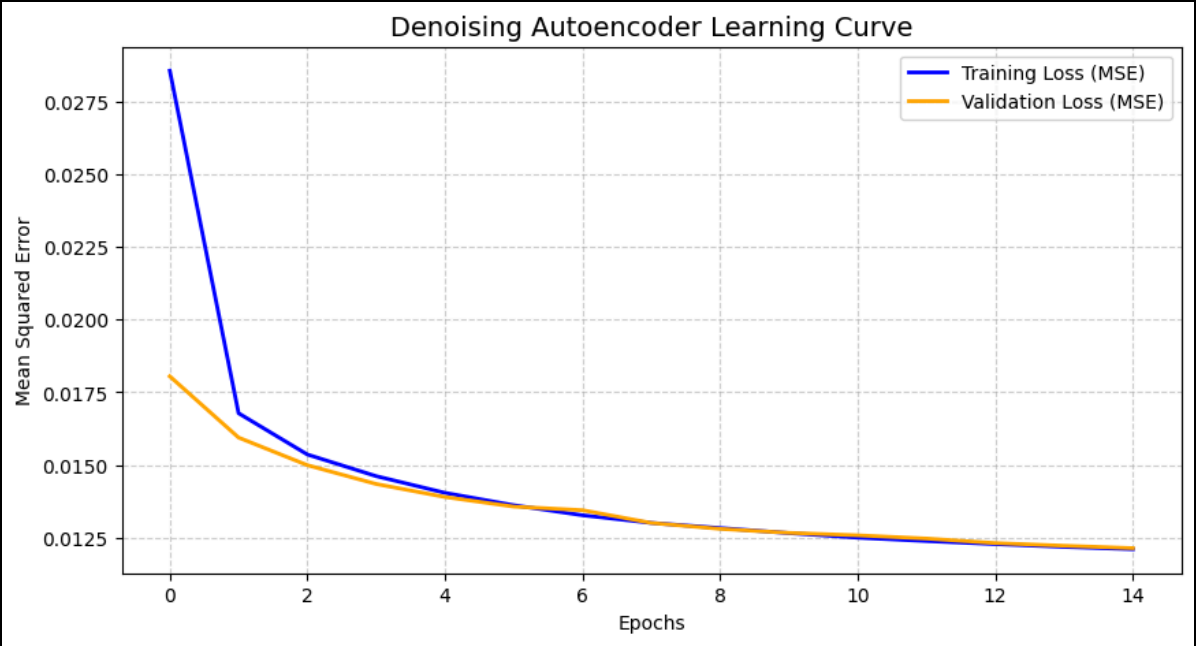
for i in range(n):
    # Original Images
    ax = plt.subplot(3, n, i + 1)
    plt.imshow(x_test[i].reshape(28, 28), cmap='gray')
    plt.title("Original", fontsize=10)
    plt.axis('off')

    # Noisy Input
    ax = plt.subplot(3, n, i + 1 + n)
    plt.imshow(x_test_noisy[i].reshape(28, 28), cmap='gray')
    plt.title("Noisy Input", fontsize=10)
    plt.axis('off')

    # Denoised Output
    ax = plt.subplot(3, n, i + 1 + 2*n)
    plt.imshow(decoded_imgs[i].reshape(28, 28), cmap='gray')
    plt.title("Denoised", fontsize=10)
    plt.axis('off')

plt.suptitle("Convolutional Autoencoder: Image Restoration Results",
             fontsize=18, fontweight='bold', y=0.95)
plt.tight_layout(rect=[0, 0.03, 1, 0.95])
plt.show()

```



=====	
PERFORMANCE EVALUATION TABLE	
=====	
MSE (Original vs Noisy):	0.0848
MSE (Original vs Denoised):	0.0121
Improvement Factor:	6.98x
=====	

6. Performance Analysis

- **Reconstruction Quality:** The model effectively removes high-frequency noise, producing smooth versions of the clothing items.
- **Visual Check:** Sneakers and Trousers show high reconstruction accuracy, while Shirts sometimes lose button details due to the smoothing effect of the MSE loss.
- **Validation Loss:** A steady decline in MSE indicates the model is successfully learning the underlying structure of the fashion items rather than the random noise.

7. Hyperparameter Tuning

Hyperparameter	Range Tested	Impact	Best Value
Noise Factor	0.2 to 0.7	Higher noise made reconstruction very blurry.	0.5
Latent Dimension	Small to Large	Smallest latent space caused high blur; larger was better for detail.	7x7x32
Optimizer	SGD, Adam	Adam converged much faster for image data.	Adam
Epochs	5 to 50	20 epochs were sufficient; beyond that, detail didn't improve much.	20

8. Conclusion

The Denoising Autoencoder successfully demonstrated the ability to recover clean images from corrupted data. By forcing the data through a convolutional bottleneck, the model learns to prioritize global patterns (the shape of a coat or shoe) over local pixel variations (noise). This experiment confirms that Autoencoders are a powerful tool for image restoration and feature extraction.

